



NumPy Fundamentals

- **NumPy** stands for **Numerical Python**.
 - It is a Python library used for working with arrays and performing numerical computations.
 - It provides support for large multi-dimensional arrays and matrices along with a large collection of high-level mathematical functions.
-



Basics of NumPy

Importing NumPy:

```
import numpy as np
```

Creating a NumPy Array:

```
arr = np.array([1, 2, 3, 4, 5])
```

Checking the Type:

```
type(arr) # <class 'numpy.ndarray'>
```



Important NumPy Properties

- **ndim**: Number of dimensions
- **shape**: Dimensions (rows, columns)
- **size**: Total number of elements
- **dtype**: Data type of elements
- **itemsize**: Size of each element in bytes

Example:

```
print(arr.ndim)    # 1 (1-D array)
print(arr.shape)   # (5,)
print(arr.dtype)   # int32 or int64
```



Array Creation Functions

- `np.zeros((2, 3))` - Creates an array filled with zeros
 - `np.ones((2, 3))` - Creates an array filled with ones
 - `np.arange(0, 10, 2)` - Creates array with values from 0 to 8, step 2
 - `np.linspace(0, 1, 5)` - 5 evenly spaced numbers between 0 and 1
-

Reshaping Arrays

Reshape changes the shape of an array without changing its data:

```
arr = np.arange(12)
new_arr = arr.reshape(3, 4) # 3 rows, 4 columns
```

Flattening Array:

```
flat_arr = new_arr.flatten()
```

Matrix Operations

Matrix Creation:

```
matrix = np.array([[1, 2], [3, 4]])
```

Matrix Transpose:

```
matrix.T
```

Matrix Addition, Subtraction, Multiplication:

```
m1 = np.array([[1, 2], [3, 4]])
m2 = np.array([[5, 6], [7, 8]])

print(m1 + m2) # Element-wise addition
print(m1 - m2) # Element-wise subtraction
print(m1 @ m2) # Matrix multiplication (new in Python 3.5+)
```

Inverse of a Matrix:

```
np.linalg.inv(matrix)
```

NumPy Array vs Python List Comparison

Feature	NumPy Array	Python List
Data Type	Homogeneous	Heterogeneous
Memory Usage	Efficient	More memory needed
Speed	Very fast	Slower
Mathematical Operations	Supported (vectorized)	Not supported directly
Functionality	Extensive (reshape, sum, mean, etc.)	Limited
Broadcasting	Supported	Not supported

Example:

```
# NumPy
import numpy as np
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print(arr1 + arr2)  # [5 7 9]

# List
list1 = [1, 2, 3]
list2 = [4, 5, 6]
print(list1 + list2)  # [1, 2, 3, 4, 5, 6]
```



Key Takeaways

- NumPy arrays are best for **numerical computation, matrix operations, and large datasets**.
- Lists are better for **general-purpose programming** with **mixed data types**.
- Always prefer **NumPy** when performance and mathematical operations are needed.